ECOMMERCE SOA BACKEND

Project Guide & Test-Folder Walkthrough

Service-Oriented Architecture · Node.js · TypeScript · Jest

Inside this document

1. Project Overview

This codebase is a complete Service-Oriented Architecture (SOA) ecommerce backend that demonstrates a production-grade multi-layer test strategy with Jest. The system is split into four bounded services — User, Product, Order, and Payment — fronted by a single Express gateway and backed by PostgreSQL.

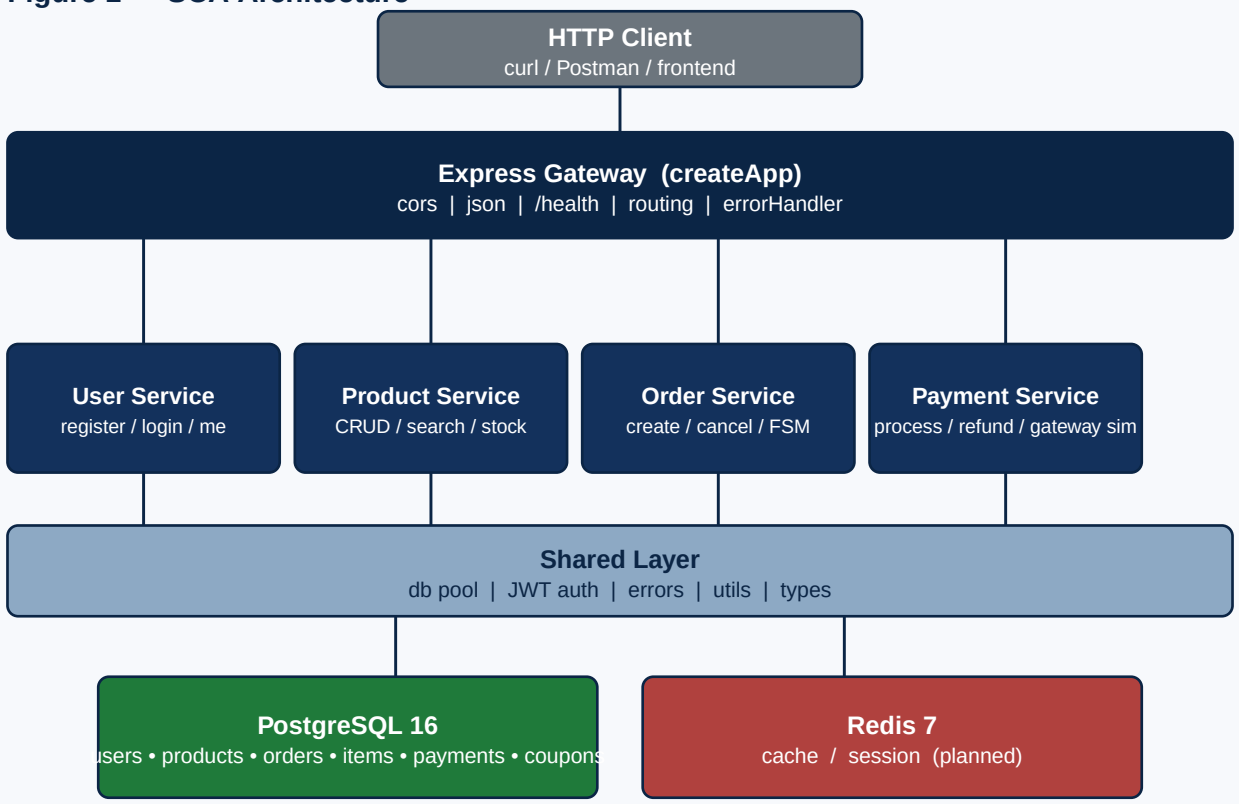
The primary goal of the repository is testing pedagogy: it is structured to show how unit, integration, and end-to-end tests differ in scope, speed, and infrastructure needs, and how Jest can run them all from a single configuration file via the multi-project feature.

Runtime	Node.js 20+, TypeScript 5.6 (strict)
HTTP	Express 4.21
Database	PostgreSQL 16 (pg pool)
Cache	Redis 7 (provisioned, reserved for future use)
Auth	JWT (jsonwebtoken) + bcryptjs password hashing
Validation	Zod schemas + custom service-level checks
Testing	Jest 29 + ts-jest + Supertest
CI	GitHub Actions (3 stages)
Container	docker-compose.test.yml (Postgres + Redis)

2. System Architecture

The system follows a layered SOA pattern. A single Express application (the Gateway) is the only HTTP entry point. Each domain service owns its routes, service-layer business logic, and a repository for data access. A Shared Layer centralises cross-cutting concerns: the pg connection pool, JWT helpers, typed error classes, and pure utility functions.

Figure 1 — SOA Architecture



2.1 Gateway Tier (src/services/gateway/app.ts)

The createApp() factory builds and returns a fully wired Express application. It mounts:

```
app.use(cors());
app.use(express.json());
app.get('/health', ...);
app.use('/api/products', productRoutes);
app.use('/api/orders',   orderRoutes);
app.use('/api/payments', paymentRoutes);
app.use('/api/auth',     userRoutes);
app.use((_req, res) => res.status(404).json({ ... }));
app.use(errorHandler);
```

Building the app via a factory (instead of side-effectful module-level code) is what makes Supertest integration and E2E tests possible — each suite calls createApp() to get a fresh in-process server.

2.2 Service Tier

Every service has three files:

routes.ts	Express router; thin layer that parses requests, calls service methods, returns JSON.
service.ts	All business logic. Receives repositories via constructor (default new instance).
repository.ts	SQL queries using the shared query()/getOne()/getMany() helpers.

Cross-service collaboration uses direct repository imports: for example, OrderService uses ProductRepository to look up products and reserve stock. This is intentional — it keeps unit tests boring (you mock the repository, not an HTTP client) while still demonstrating cross-service coupling.

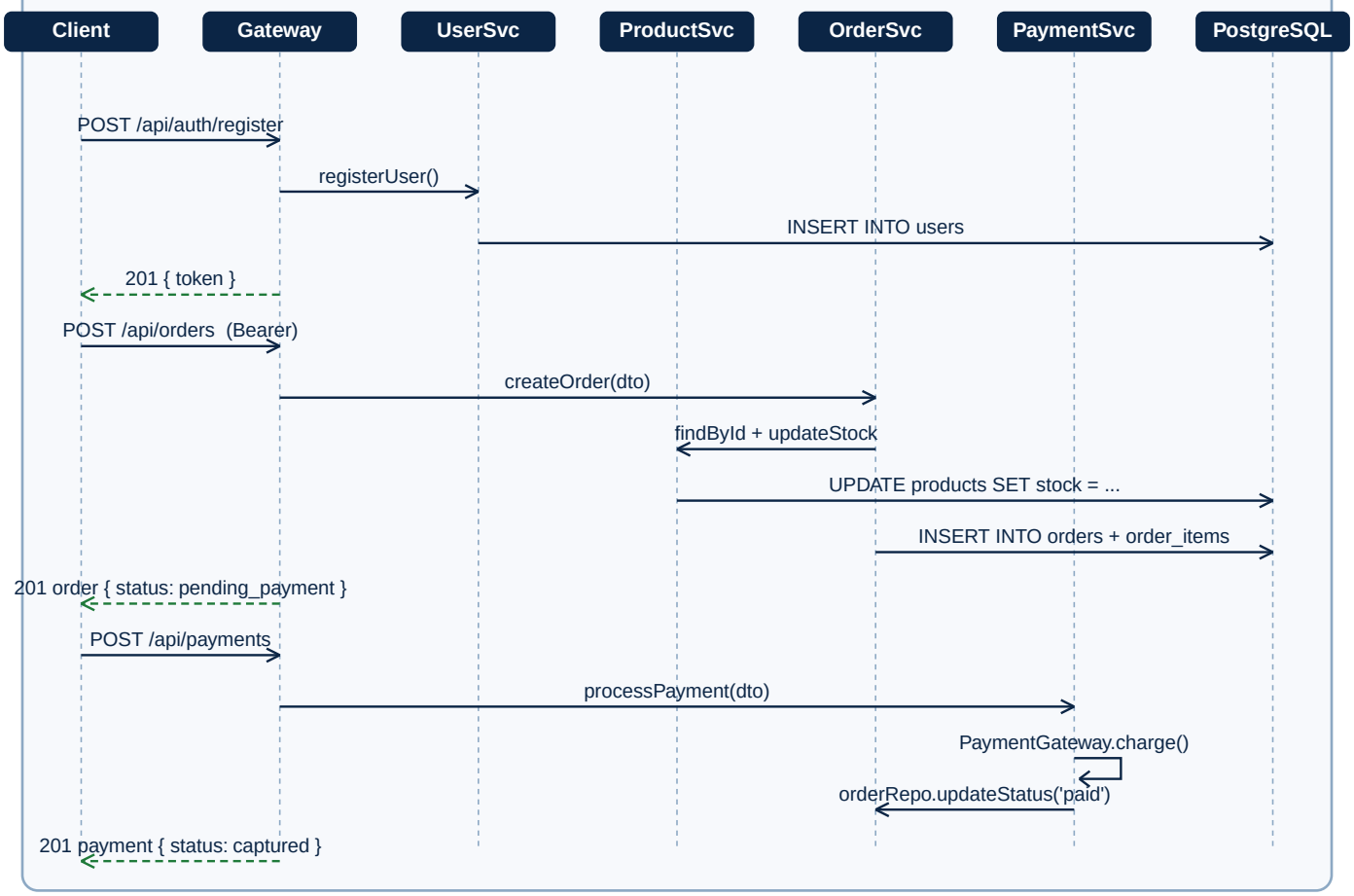
2.3 Shared Layer

database/index.ts	getPool(), query(), schema init, truncateAllTables().
middleware/auth.ts	generateToken(), requireAuth, requireAdmin.
middleware/errorHandler.ts	Typed error classes mapped to HTTP status codes.
utils/index.ts	Pure helpers: calculateTax, applyDiscount, paginate, generateId.
types/index.ts	Shared TS interfaces (User, Product, Order, Payment, DTOs).

3. Request Lifecycle — Checkout Flow

The diagram below traces a full checkout: register, place an order, then pay. It shows precisely how the gateway dispatches into per-service modules and how those modules collaborate with the shared database layer.

Figure 2 — Checkout Sequence (E2E)



Two important invariants emerge from the diagram:

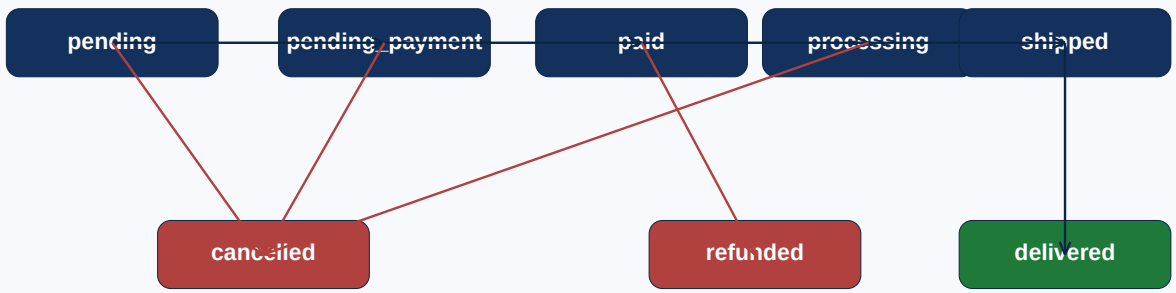
- Stock is decremented before the order row is written, so concurrent orders cannot oversell. The atomicity comes from the SQL UPDATE products SET stock = stock - $1 WHERE id = $2 AND stock >= $1 RETURNING * pattern.
- Payment is only attempted when the order is in pending_payment, and a gateway success transitions both records (payment !captured, order !paid) in two writes.

4. Domain Model

Six tables form the persistence layer. They are created by initializeDatabase() and dropped/truncated by sibling helpers in src/shared/database/index.ts.

users	id, email, password_hash, first_name, last_name, role, is_active, timestamps
products	id, name, description, price, stock, category, timestamps (CHECK price"e0, stock"e0)
orders	id, user_id (FK), subtotal, tax, discount, total, status, shipping_*, payment_id, coupon_code
order_items	id, order_id (FK CASCADE), product_id, product_name, quantity, unit_price, total_price
payments	id, order_id (FK), user_id (FK), amount, currency, method, status, transaction_id, failure_reason
coupons	id, code (UNIQUE), discount_type, discount_value, min_order_amount, max_uses, current_uses, expires_at

Figure 4 — Order State Machine

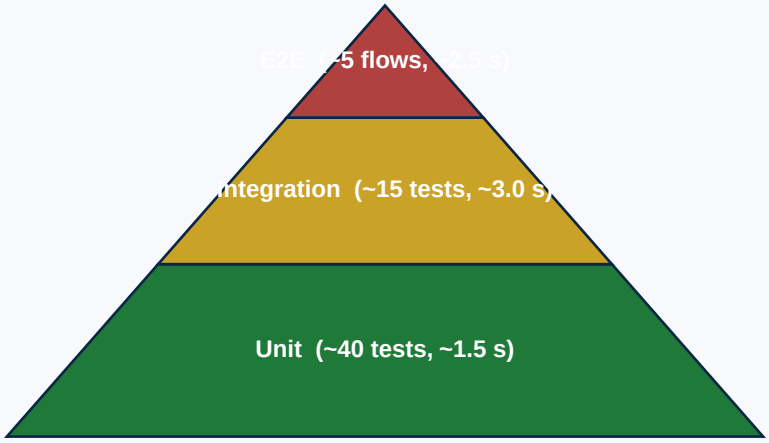


The state machine is enforced inside OrderService.updateOrderStatus(). Each source state has an explicit list of allowed destinations; invalid transitions throw ValidationError. Cancellation rolls back reserved stock by incrementing the product rows.

5. Test Strategy

The project layers tests in three rings — unit, integration, end-to-end — and uses Jest's multi-project feature to run them with different setups from a single configuration. The diagram below shows the rough volume and runtime distribution.

Figure 3 — Test Pyramid



5.1 Unit Tests tests/unit/

Scope	Single method or pure function.
Database	Fully mocked via jest.mock() on the repository or shared/database module.
Speed	~30–80 ms per test.
External I/O	None. PaymentGateway is itself a class so tests inject a mocked instance.
Files	5 spec files across 4 service folders.

What is exercised

- product.service.test.ts — listing & pagination math; category filter; ILIKE search with min-length validation; create/update/delete; reserveStock decrement; rejection of negative price/stock and empty category.
- order.service.test.ts — totals computation; stock reservation; status FSM transitions (valid + invalid); stock release on cancellation; terminal-state guard.
- payment.service.test.ts — successful capture; declined card (tok_fail); insufficient funds; amount mismatch vs order total; refund of captured payment; refund of non-captured rejected. Includes direct tests of the PaymentGateway simulator.
- user.service.test.ts — bcrypt hashing, JWT signing, duplicate email rejection, login with wrong password.

Pattern

```
jest.mock('../../../src/services/order-service/order.repository');
jest.mock('../../../src/services/product-service/product.repository');

const mockOrderRepo   = new OrderRepository() as jest.Mocked<OrderRepository>;
const mockProductRepo = new ProductRepository() as jest.Mocked<ProductRepository>;
const service = new OrderService(mockOrderRepo, mockProductRepo);

mockProductRepo.findById.mockResolvedValueOnce({ id: 'p1', stock: 50, price: 100 });
mockOrderRepo.create.mockResolvedValueOnce({ id: 'o1', total: 270, status:
'pending_payment' });
```

5.2 Integration Tests tests/integration/

Scope	Real HTTP via Supertest, real PostgreSQL, single endpoint at a time.
Database	Test container on port 5433. Schema initialised once per file; tables truncated before each test.
Speed	~150–250 ms per test.
Lifecycle	integration.globalSetup verifies connectivity once; integration.setup runs initializeDatabase + truncateAllTables per file/test.

What is exercised

- auth-api.test.ts — POST /api/auth/register happy path returning a JWT; duplicate-email rejection; short-password and invalid-email validation errors; POST /api/auth/login success and wrong-password rejection.
- product-api.test.ts — full CRUD against the real DB: list empty, list after seed, category filter, paginated retrieval (15 rows split into 2 pages), POST as admin succeeds, POST as customer is 403, POST without auth is 401, GET by id 404 for nonexistent UUID, PUT updates price, DELETE removes a row and subsequent GET returns 404, /search?q=keyboard performs an ILIKE search.

Pattern

```
import request from 'supertest';
import { createApp } from '../../src/services/gateway/app';
import { query } from '../../src/shared/database';
import { generateAdminToken } from '../fixtures/testData';

const app = createApp();
const adminToken = generateAdminToken();

it('creates a product with admin token', async () => {
  const res = await request(app)
    .post('/api/products')
    .set('Authorization', `Bearer ${adminToken}`)
    .send(testProducts[0]);
  expect(res.status).toBe(201);
});
```

5.3 End-to-End Tests tests/e2e/

Scope	Multi-service business flows that span the whole stack.
Database	Same test Postgres as integration; truncated between tests.
Speed	~400–700 ms per flow (multiple HTTP requests each).
Files	checkout-flow.test.ts, refund-flow.test.ts.

checkout-flow.test.ts — the canonical happy path

- Step 1 ≡ public browse: GET /api/products returns ≥3 items.
- Step 2 ≡ register + login: mints a JWT for a brand-new customer.
- Step 3 ≡ create order with two items and a shipping address. Asserts subtotal math and status === 'pending_payment'.
- Step 4 ≡ verify stock was decremented by the ordered quantity.
- Step 5 ≡ process payment with tok_visa. Asserts 201, status: 'succeeded', and a transactionId.
- Step 6 ≡ re-fetch the order: status flipped to 'paid', paidAt populated.
- Step 7 ≡ list user orders: the new order is present.

Negative flows in the same file

- Payment failure with tok_fail: 402 + order remains pending_payment.
- Cancelling an order releases stock back to the product row.
- Customer B cannot read Customer A's order: 403.

refund-flow.test.ts

Builds a captured payment via the API, then exercises POST /api/payments/:id/refund: a captured payment can be refunded, non-captured payments cannot, and the parent order status flips to refunded.

6. Test Infrastructure

6.1 Jest Multi-Project Config

jest.config.ts defines six projects. Each testMatch targets a specific folder so npm run test:unit selects only unit projects, etc. Crucially, only the integration and e2e projects load global setup/teardown hooks — unit projects stay fast because they never touch a real DB.

```
projects: [
  { displayName: 'unit:product', testMatch: ['<rootDir>/tests/unit/product-service/**/*.test.ts'] },
  { displayName: 'unit:order',   testMatch: ['<rootDir>/tests/unit/order-service/**/*.test.ts'] },
  { displayName: 'unit:payment', testMatch: ['<rootDir>/tests/unit/payment-service/**/*.test.ts'] },
  { displayName: 'unit:user',    testMatch: ['<rootDir>/tests/unit/user-service/**/*.test.ts'] },
  { displayName: 'integration',  testMatch: ['<rootDir>/tests/integration/**/*.test.ts'],
    globalSetup:    '<rootDir>/tests/setup/integration.globalSetup.ts',
    globalTeardown: '<rootDir>/tests/setup/integration.globalTeardown.ts',
    setupFilesAfterSetup: ['<rootDir>/tests/setup/integration.setup.ts'] },
  { displayName: 'e2e',          testMatch: ['<rootDir>/tests/e2e/**/*.test.ts'],
    globalSetup:    '<rootDir>/tests/setup/e2e.globalSetup.ts',
    globalTeardown: '<rootDir>/tests/setup/e2e.globalTeardown.ts',
    setupFilesAfterSetup: ['<rootDir>/tests/setup/e2e.setup.ts'] },
]
coverageThresholds: { global: { branches: 70, functions: 75, lines: 80, statements: 80 } }
```

6.2 Setup & Teardown Files

unit.setup.ts	Silences console.log/error so unit-test output stays readable. Restored in afterAll.
integration.globalSetup.ts	One-time DB reachability check; sets env defaults (DB_HOST, DB_PORT 5433, DB_NAME, JWT_SECRET). Throws with a helpful 'docker compose up' hint if Postgres is unreachable.
integration.setup.ts	Per-file: beforeAll !InitializeDatabase(); beforeEach !truncateAllTables(); afterAll !' closePool().
integration.globalTeardown.ts	Closes any lingering connections after the Jest run.
e2e.globalSetup.ts	Sets the same env defaults as integration, plus NODE_ENV=test.
e2e.setup.ts	Same per-file truncate pattern as integration.
e2e.globalTeardown.ts	Final cleanup.

6.3 Fixtures tests/fixtures/testData.ts

All shared test data lives here so specs stay declarative. Exports include:

- testUsers.admin/.customer/.customer2 — ready-to-POST registration payloads.
- generateAdminToken(), generateCustomerToken(), generateTokenForUser(payload) — signed JWTs without going through the register endpoint.
- invalidProducts, invalidUsers — precomputed payloads for negative tests (missing name, negative price, short password, invalid email …).

6.4 Database Mock tests/mocks/database.mock.ts

Replaces the real pg pool with jest.fn()s. Provides three primitive mocks (mockQuery, mockGetOne, mockGetMany) and four ergonomic helpers:

mockQueryReturning(row)	Stubs an INSERT ... RETURNING * with rows:[row] and rowCount:1.
mockQueryCount(n)	Stubs a SELECT COUNT(*) with rows:[{count:n}].
mockQueryDelete(ok)	Stubs a DELETE/UPDATE by setting rowCount to 1 or 0.
resetDbMocks()	Resets the three mocks. Call from beforeEach.

7. CI/CD Pipeline

GitHub Actions runs three jobs that mirror the local npm scripts. Splitting unit tests into their own job gives developers fast PR feedback even when the integration job is queued behind a slow container start.

1. unit-tests	Checkout !šetup Node !hpm ci !hpm run test:unit. No services. Typical runtime: ~30 s.
2. integration-e2e-tests	Defines a services.postgres container (image postgres:16) with healthcheck. Runs test:integration then test:e2e against it. Uses env vars matching integration.globalSetup.ts.
3. coverage	Re-runs everything with --coverage --ci --reporters=jest-junit. Uploads coverage/lcov.info and junit.xml as artifacts. Optional step posts a summary comment on the PR.

7.1 Local equivalent

```
docker compose -f docker-compose.test.yml up -d --wait

npm run test:unit         # ~1.5 s — no DB
npm run test:integration  # ~3.0 s — real DB
npm run test:e2e          # ~2.5 s — full flows
npm run test:coverage     # everything + lcov + json-summary

docker compose -f docker-compose.test.yml down -v
```

Appendix A — Full File Index

Source files

```
src/index.ts
src/services/gateway/app.ts
src/services/user-service/{user.routes,user.service,user.repository}.ts
src/services/product-service/{product.routes,product.service,product.repository}.ts
src/services/order-service/{order.routes,order.service,order.repository}.ts
src/services/payment-service/{payment.routes,payment.service,payment.repository}.ts
src/shared/database/index.ts
src/shared/middleware/{auth,errorHandler}.ts
src/shared/types/index.ts
src/shared/utils/index.ts
```

Test files

```
tests/unit/product-service/product.service.test.ts
tests/unit/product-service/utils.test.ts
tests/unit/order-service/order.service.test.ts
tests/unit/payment-service/payment.service.test.ts
tests/unit/user-service/user.service.test.ts
tests/integration/auth-api.test.ts
tests/integration/product-api.test.ts
tests/e2e/checkout-flow.test.ts
tests/e2e/refund-flow.test.ts
tests/fixtures/testData.ts
tests/mocks/database.mock.ts
tests/setup/unit.setup.ts
tests/setup/integration.globalSetup.ts
tests/setup/integration.globalTeardown.ts
tests/setup/integration.setup.ts
tests/setup/e2e.globalSetup.ts
tests/setup/e2e.globalTeardown.ts
tests/setup/e2e.setup.ts
```

Build & config

```
jest.config.ts
tsconfig.json
docker-compose.test.yml
package.json
```

Appendix B — Glossary

SOA	Service-Oriented Architecture — modules are organised by business capability behind a single gateway.
Repository	Thin class wrapping all SQL for an aggregate. Mocked in unit tests.
DTO	Data Transfer Object — request/response shapes defined in shared/types.
Supertest	HTTP testing library — wraps Express app, returns chainable request builder.

jest.mock()	Auto-mocks a module so every export becomes a jest.fn(). Used to neutralise repositories & DB.
truncate vs drop	Truncate is much faster than DROP+CREATE; we use it between each integration/E2E test.
State machine	Order status transitions enforced explicitly in OrderService.updateOrderStatus().
Test pyramid	Convention where unit tests outnumber integration tests, which outnumber E2E tests.